

# Capstone Project: 1

## Supply Chain Monitoring and Optimization Platform

**Objective:** Build a practical system to track orders, inventory, and shipments using real-world tools like MySQL, MongoDB, Python, PySpark, Azure Databricks, and Azure DevOps.

### Week 1 – Database Foundations: MySQL & MongoDB

#### Tools: MySQL, MongoDB

#### Capstone Tasks

1. Create MySQL tables for orders, suppliers, inventory

#### Create Database

```
CREATE DATABASE supply_chain_db;  
USE supply_chain_db;
```

#### Create Suppliers Table

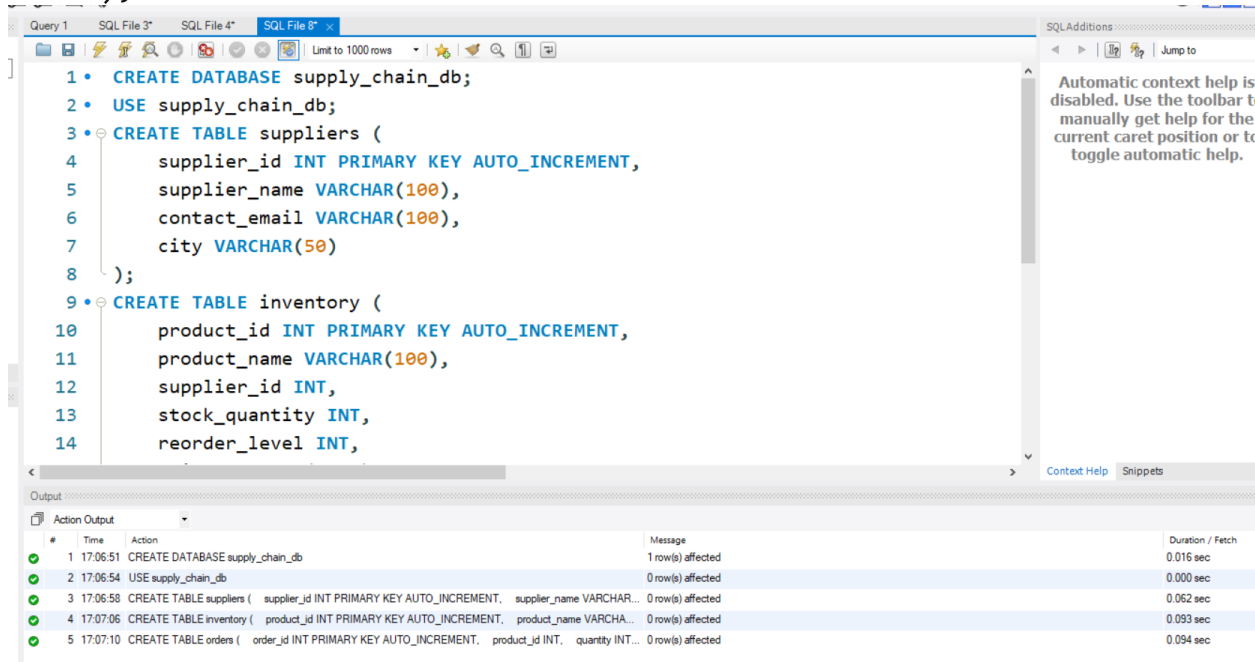
```
CREATE TABLE suppliers (  
    supplier_id INT PRIMARY KEY AUTO_INCREMENT,  
    supplier_name VARCHAR(100),  
    contact_email VARCHAR(100),  
    city VARCHAR(50)  
);
```

#### Create Inventory Table

```
CREATE TABLE inventory (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100),  
    supplier_id INT,  
    stock_quantity INT,  
    reorder_level INT,  
    price DECIMAL(10,2),  
    FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)  
);
```

## Create Orders Table

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_id INT,  
    quantity INT,  
    order_date DATE,  
    delivery_date DATE,  
    status VARCHAR(30),  
    FOREIGN KEY (product_id) REFERENCES inventory(product_id)  
);
```



## Insert Data

### Suppliers

```
INSERT INTO suppliers (supplier_name, contact_email, city) VALUES  
( 'ABC Traders', 'abc@gmail.com', 'Chennai'),  
( 'Global Supplies', 'global@gmail.com', 'Bangalore'),  
( 'Tech Source Ltd', 'techsource@gmail.com', 'Hyderabad'),  
( 'Prime Electronics', 'prime@gmail.com', 'Mumbai'),  
( 'Smart Distributors', 'smart@gmail.com', 'Pune');
```

## Inventory

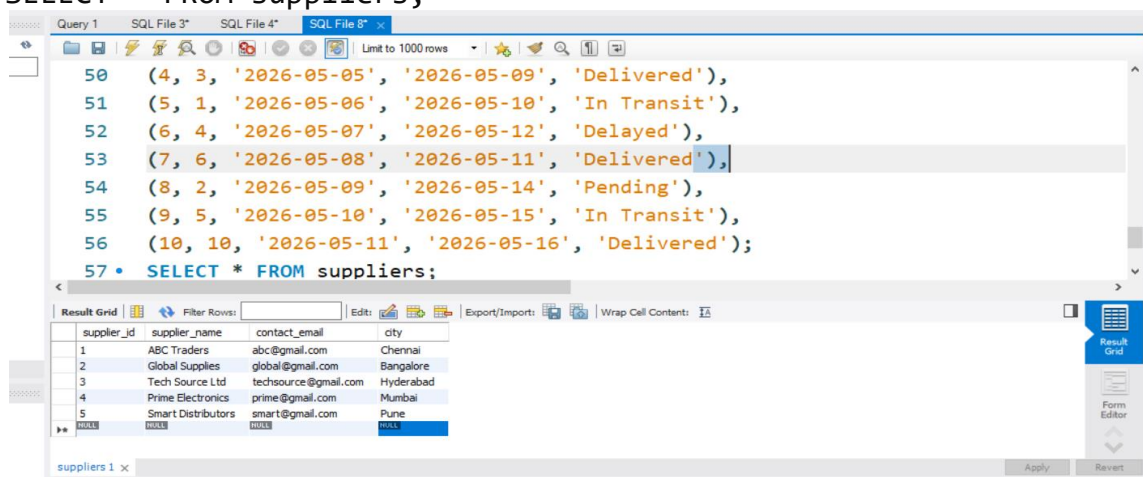
```
INSERT INTO inventory (product_name, supplier_id, stock_quantity,
reorder_level, price) VALUES
('Laptop', 1, 50, 10, 55000),
('Mouse', 2, 100, 20, 500),
('Keyboard', 1, 15, 10, 1200),
('Monitor', 3, 40, 10, 12000),
('Printer', 4, 25, 5, 15000),
('Scanner', 5, 18, 5, 8000),
('Webcam', 2, 60, 15, 2500),
('Speaker', 3, 35, 10, 3000),
('Hard Disk', 4, 20, 5, 4500),
('USB Drive', 5, 150, 30, 700);
```

## Orders

```
INSERT INTO orders (product_id, quantity, order_date,
delivery_date, status) VALUES
(1, 5, '2026-05-01', '2026-05-05', 'Delivered'),
(2, 20, '2026-05-02', '2026-05-08', 'Delayed'),
(3, 2, '2026-05-04', '2026-05-06', 'Pending'),
(4, 3, '2026-05-05', '2026-05-09', 'Delivered'),
(5, 1, '2026-05-06', '2026-05-10', 'In Transit'),
(6, 4, '2026-05-07', '2026-05-12', 'Delayed'),
(7, 6, '2026-05-08', '2026-05-11', 'Delivered'),
(8, 2, '2026-05-09', '2026-05-14', 'Pending'),
(9, 5, '2026-05-10', '2026-05-15', 'In Transit'),
(10, 10, '2026-05-11', '2026-05-16', 'Delivered');
```

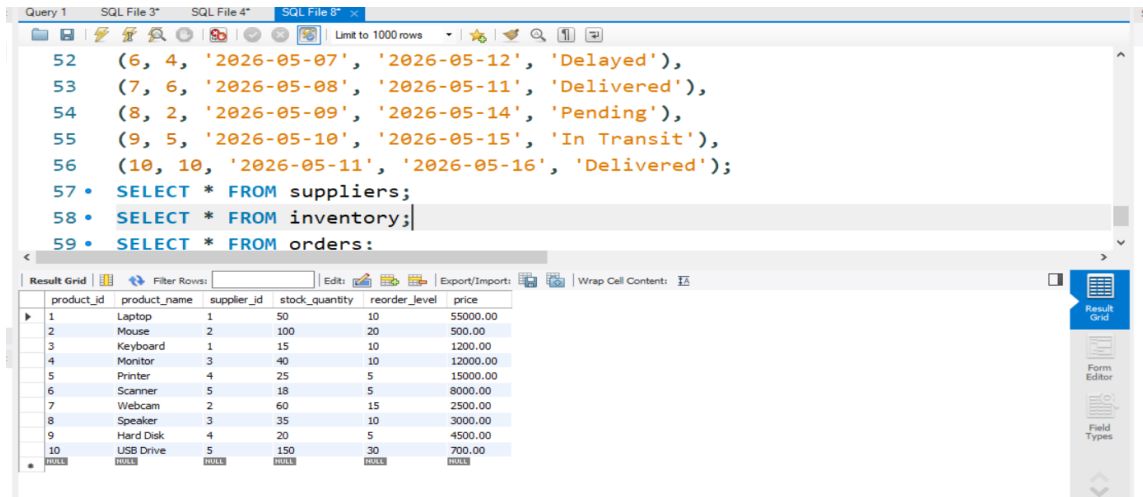
## Check Data

```
SELECT * FROM suppliers;
```



| supplier_id | supplier_name      | contact_email        | city      |
|-------------|--------------------|----------------------|-----------|
| 1           | ABC Traders        | abc@gmail.com        | Chennai   |
| 2           | Global Supplies    | global@gmail.com     | Bangalore |
| 3           | Tech Source Ltd    | techsource@gmail.com | Hyderabad |
| 4           | Prime Electronics  | prime@gmail.com      | Mumbai    |
| 5           | Smart Distributors | smart@gmail.com      | Pune      |

SELECT \* FROM inventory;



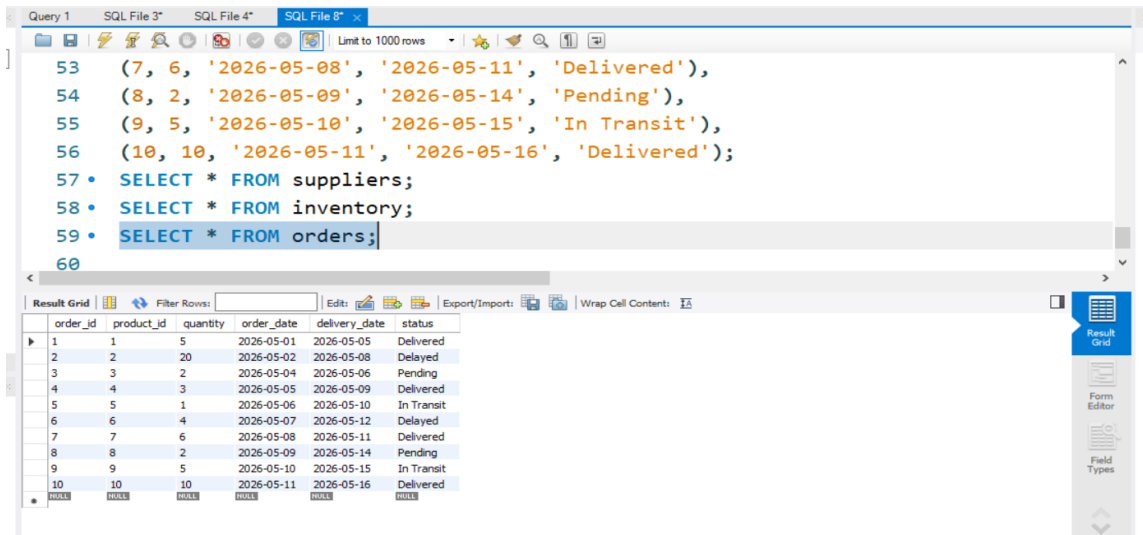
The screenshot shows a SQL IDE interface. The query window contains the following SQL statements:

```
52 (6, 4, '2026-05-07', '2026-05-12', 'Delayed'),
53 (7, 6, '2026-05-08', '2026-05-11', 'Delivered'),
54 (8, 2, '2026-05-09', '2026-05-14', 'Pending'),
55 (9, 5, '2026-05-10', '2026-05-15', 'In Transit'),
56 (10, 10, '2026-05-11', '2026-05-16', 'Delivered');
57 • SELECT * FROM suppliers;
58 • SELECT * FROM inventory;
59 • SELECT * FROM orders;
```

The result grid displays the data for the 'inventory' table:

| product_id | product_name | supplier_id | stock_quantity | reorder_level | price    |
|------------|--------------|-------------|----------------|---------------|----------|
| 1          | Laptop       | 1           | 50             | 10            | 55000.00 |
| 2          | Mouse        | 2           | 100            | 20            | 500.00   |
| 3          | Keyboard     | 1           | 15             | 10            | 1200.00  |
| 4          | Monitor      | 3           | 40             | 10            | 12000.00 |
| 5          | Printer      | 4           | 25             | 5             | 15000.00 |
| 6          | Scanner      | 5           | 18             | 5             | 8000.00  |
| 7          | Webcam       | 2           | 60             | 15            | 2500.00  |
| 8          | Speaker      | 3           | 35             | 10            | 3000.00  |
| 9          | Hard Disk    | 4           | 20             | 5             | 4500.00  |
| 10         | USB Drive    | 5           | 150            | 30            | 700.00   |

SELECT \* FROM orders;



The screenshot shows a SQL IDE interface. The query window contains the following SQL statements:

```
53 (7, 6, '2026-05-08', '2026-05-11', 'Delivered'),
54 (8, 2, '2026-05-09', '2026-05-14', 'Pending'),
55 (9, 5, '2026-05-10', '2026-05-15', 'In Transit'),
56 (10, 10, '2026-05-11', '2026-05-16', 'Delivered');
57 • SELECT * FROM suppliers;
58 • SELECT * FROM inventory;
59 • SELECT * FROM orders;
60
```

The result grid displays the data for the 'orders' table:

| order_id | product_id | quantity | order_date | delivery_date | status     |
|----------|------------|----------|------------|---------------|------------|
| 1        | 1          | 5        | 2026-05-01 | 2026-05-05    | Delivered  |
| 2        | 2          | 20       | 2026-05-02 | 2026-05-08    | Delayed    |
| 3        | 3          | 2        | 2026-05-04 | 2026-05-06    | Pending    |
| 4        | 4          | 3        | 2026-05-05 | 2026-05-09    | Delivered  |
| 5        | 5          | 1        | 2026-05-06 | 2026-05-10    | In Transit |
| 6        | 6          | 4        | 2026-05-07 | 2026-05-12    | Delayed    |
| 7        | 7          | 6        | 2026-05-08 | 2026-05-11    | Delivered  |
| 8        | 8          | 2        | 2026-05-09 | 2026-05-14    | Pending    |
| 9        | 9          | 5        | 2026-05-10 | 2026-05-15    | In Transit |
| 10       | 10         | 10       | 2026-05-11 | 2026-05-16    | Delivered  |

## 2. Perform basic CRUD operations

### CRUD Operations

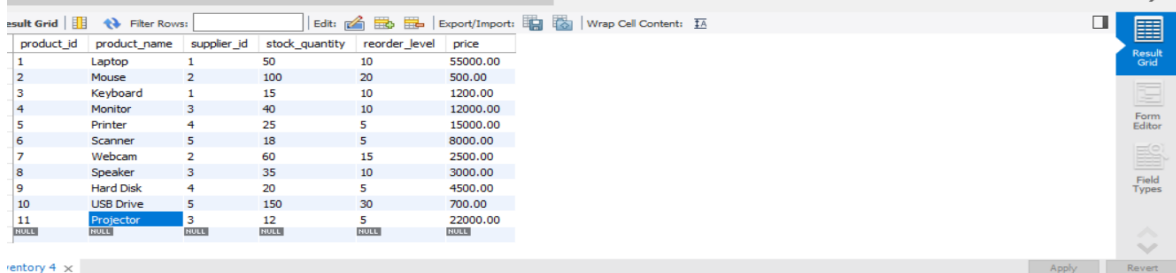
#### 1. CREATE – Insert New Product

```
INSERT INTO inventory (product_name, supplier_id, stock_quantity,  
reorder_level, price)  
VALUES ('Projector', 3, 12, 5, 22000);
```

```

58 • SELECT * FROM inventory;
59 • SELECT * FROM orders;
60 • INSERT INTO inventory (product_name, supplier_id, stock_quantity, reorder_level,
61 VALUES ('Projector', 3, 12, 5, 22000);
62 • select * from inventory;
63

```



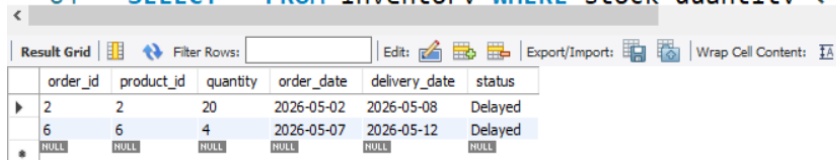
| product_id | product_name | supplier_id | stock_quantity | reorder_level | price    |
|------------|--------------|-------------|----------------|---------------|----------|
| 1          | Laptop       | 1           | 50             | 10            | 55000.00 |
| 2          | Mouse        | 2           | 100            | 20            | 500.00   |
| 3          | Keyboard     | 1           | 15             | 10            | 1200.00  |
| 4          | Monitor      | 3           | 40             | 10            | 12000.00 |
| 5          | Printer      | 4           | 25             | 5             | 15000.00 |
| 6          | Scanner      | 5           | 18             | 5             | 8000.00  |
| 7          | Webcam       | 2           | 60             | 15            | 2500.00  |
| 8          | Speaker      | 3           | 35             | 10            | 3000.00  |
| 9          | Hard Disk    | 4           | 20             | 5             | 4500.00  |
| 10         | USB Drive    | 5           | 150            | 30            | 700.00   |
| 11         | Projector    | 3           | 12             | 5             | 22000.00 |

## 2. READ – Query Data

```

SELECT * FROM inventory;
SELECT * FROM orders WHERE status = 'Delayed';
63 • SELECT * FROM orders WHERE status = 'Delayed';
64 • SELECT * FROM inventory WHERE stock_quantity <= reorder_level;

```

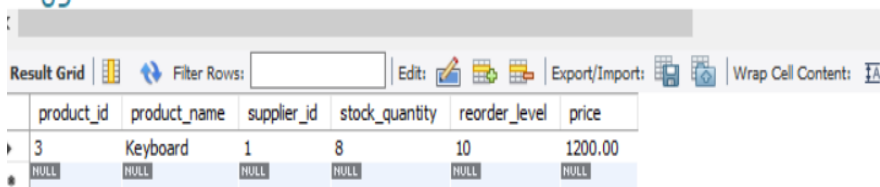


| order_id | product_id | quantity | order_date | delivery_date | status  |
|----------|------------|----------|------------|---------------|---------|
| 2        | 2          | 20       | 2026-05-02 | 2026-05-08    | Delayed |
| 6        | 6          | 4        | 2026-05-07 | 2026-05-12    | Delayed |
| *        | NULL       | NULL     | NULL       | NULL          | NULL    |

```

SELECT * FROM inventory WHERE stock_quantity <=
reorder_level;
63 • SELECT * FROM orders WHERE status = 'Delayed';
64 • SELECT * FROM inventory WHERE stock_quantity <= reorder_level;
65

```



| product_id | product_name | supplier_id | stock_quantity | reorder_level | price   |
|------------|--------------|-------------|----------------|---------------|---------|
| 3          | Keyboard     | 1           | 8              | 10            | 1200.00 |
| *          | NULL         | NULL        | NULL           | NULL          | NULL    |

## 3. UPDATE – Modify Records

```

UPDATE inventory SET stock_quantity = 80 WHERE product_id = 1;
UPDATE orders SET status = 'Delivered' WHERE order_id = 2;

```

```

65 • UPDATE inventory SET stock_quantity = 80 WHERE product_id = 1;
66 • UPDATE orders SET status = 'Delivered' WHERE order_id = 2;
67 |
68

```

| Output        |          |                                                               |                                                          |  |
|---------------|----------|---------------------------------------------------------------|----------------------------------------------------------|--|
| Action Output |          |                                                               |                                                          |  |
| #             | Time     | Action                                                        | Message                                                  |  |
| ✓ 20          | 17:22:55 | UPDATE inventory SET stock_quantity = 80 WHERE product_id = 1 | 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 |  |
| ✓ 21          | 17:23:00 | UPDATE orders SET status = 'Delivered' WHERE order_id = 2     | 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 |  |

#### 4. DELETE – Remove Records

```
DELETE FROM orders WHERE order_id = 10;
```

```

67 • DELETE FROM orders WHERE order_id = 10;
68 |

```

| Output        |          |                                                           |                                                          |  |
|---------------|----------|-----------------------------------------------------------|----------------------------------------------------------|--|
| Action Output |          |                                                           |                                                          |  |
| #             | Time     | Action                                                    | Message                                                  |  |
| ✓ 21          | 17:23:00 | UPDATE orders SET status = 'Delivered' WHERE order_id = 2 | 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 |  |
| ✓ 22          | 17:23:57 | DELETE FROM orders WHERE order_id = 10                    | 1 row(s) affected                                        |  |

#### 3. Write stored procedures (e.g., auto reorder trigger)

##### Auto Reorder Trigger

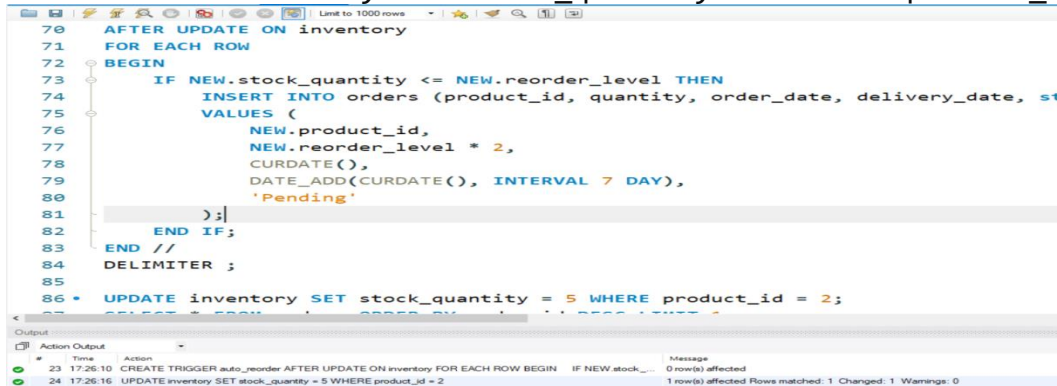
```

DELIMITER //
CREATE TRIGGER auto_reorder
AFTER UPDATE ON inventory
FOR EACH ROW
BEGIN
    IF NEW.stock_quantity <= NEW.reorder_level THEN
        INSERT INTO orders (product_id, quantity, order_date,
        delivery_date, status)
        VALUES (
            NEW.product_id,
            NEW.reorder_level * 2,
            CURDATE(),
            DATE_ADD(CURDATE(), INTERVAL 7 DAY),
            'Pending'
        );
    END IF;
END //
DELIMITER ;

```

-- Test the trigger (stock drops below reorder level, auto-order fires):

UPDATE inventory SET stock\_quantity = 5 WHERE product\_id = 2;



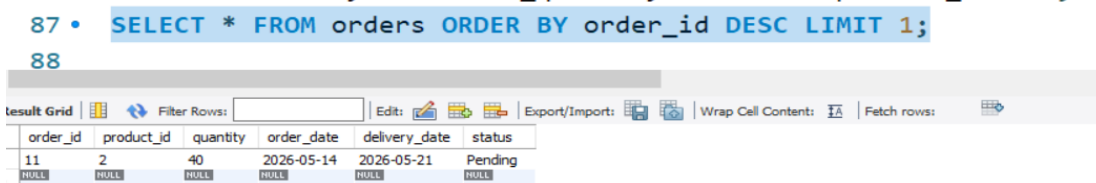
The screenshot shows a SQL IDE with a trigger definition and its execution output. The trigger is named 'auto\_reorder' and is defined as an AFTER UPDATE trigger on the 'inventory' table. It fires for each row when the stock quantity drops below the reorder level. The trigger logic is as follows:

```
70 AFTER UPDATE ON inventory
71 FOR EACH ROW
72 BEGIN
73     IF NEW.stock_quantity <= NEW.reorder_level THEN
74         INSERT INTO orders (product_id, quantity, order_date, delivery_date, status)
75         VALUES (
76             NEW.product_id,
77             NEW.reorder_level * 2,
78             CURDATE(),
79             DATE_ADD(CURDATE(), INTERVAL 7 DAY),
80             'Pending'
81         );
82     END IF;
83 END //
84 DELIMITER ;
```

The output window shows the following actions:

| #  | Time     | Action                                                                                                                                                                                                                                                                                                                          | Message                                                  |
|----|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 23 | 17:26:10 | CREATE TRIGGER auto_reorder AFTER UPDATE ON inventory FOR EACH ROW BEGIN IF NEW.stock_quantity <= NEW.reorder_level THEN INSERT INTO orders (product_id, quantity, order_date, delivery_date, status) VALUES (NEW.product_id, NEW.reorder_level * 2, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), 'Pending'); END IF; END // | 0 row(s) affected                                        |
| 24 | 17:26:16 | UPDATE inventory SET stock_quantity = 5 WHERE product_id = 2                                                                                                                                                                                                                                                                    | 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 |

SELECT \* FROM orders ORDER BY order\_id DESC LIMIT 1;



The screenshot shows a SQL IDE with a query and its result grid. The query is:

```
87 • SELECT * FROM orders ORDER BY order_id DESC LIMIT 1;
88
```

The result grid shows the following data:

| order_id | product_id | quantity | order_date | delivery_date | status  |
|----------|------------|----------|------------|---------------|---------|
| 11       | 2          | 40       | 2026-05-14 | 2026-05-21    | Pending |
| HULL     | HULL       | HULL     | HULL       | HULL          | HULL    |

## 4. Store shipment logs in MongoDB

### MongoDB – Shipment Logs

#### Create Database & Collection

use supply\_chain\_logs

#### Insert Shipment Logs

```
db.shipment_logs.insertMany([
  { order_id: 1, shipment_status: 'In Transit', location: 'Chennai Hub', timestamp: new Date() },
  { order_id: 2, shipment_status: 'Delayed', location: 'Bangalore Warehouse', timestamp: new Date() },
  { order_id: 3, shipment_status: 'Delivered', location: 'Hyderabad', timestamp: new Date() },
  { order_id: 4, shipment_status: 'Pending', location: 'Mumbai Hub', timestamp: new Date() },
  { order_id: 5, shipment_status: 'In Transit', location: 'Pune Warehouse', timestamp: new Date() }
])
```

```

> db.shipment_logs.insertMany([
  { order_id: 1, shipment_status: 'In Transit', location: 'Chennai Hub', timestamp: new Date() },
  { order_id: 2, shipment_status: 'Delayed', location: 'Bangalore Warehouse', timestamp: new Date() },
  { order_id: 3, shipment_status: 'Delivered', location: 'Hyderabad', timestamp: new Date() },
  { order_id: 4, shipment_status: 'Pending', location: 'Mumbai Hub', timestamp: new Date() },
  { order_id: 5, shipment_status: 'In Transit', location: 'Pune Warehouse', timestamp: new Date() }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a05dd2159f4ba6f85d668fa'),
    '1': ObjectId('6a05dd2159f4ba6f85d668fb'),
    '2': ObjectId('6a05dd2159f4ba6f85d668fc'),
    '3': ObjectId('6a05dd2159f4ba6f85d668fd'),
    '4': ObjectId('6a05dd2159f4ba6f85d668fe')
  }
}

```

## View, Update, Delete

### VIEW

db.shipment\_logs.find()

```

> db.shipment_logs.find()
< {
  _id: ObjectId('6a05dd2159f4ba6f85d668fa'),
  order_id: 1,
  shipment_status: 'In Transit',
  location: 'Chennai Hub',
  timestamp: 2026-05-14T14:33:05.250Z
}
{
  _id: ObjectId('6a05dd2159f4ba6f85d668fb'),
  order_id: 2,
  shipment_status: 'Delayed',
  location: 'Bangalore Warehouse',
  timestamp: 2026-05-14T14:33:05.250Z
}
{
  _id: ObjectId('6a05dd2159f4ba6f85d668fc'),
  order_id: 3,
  shipment_status: 'Delivered',
  location: 'Hyderabad',
  timestamp: 2026-05-14T14:33:05.250Z
}

```



## UPDATE

```
db.shipment_logs.updateOne(  
  { order_id: 2 },  
  { $set: { shipment_status: 'In Transit' } }  
)
```

```
}  
> db.shipment_logs.updateOne(  
  { order_id: 2 },  
  { $set: { shipment_status: 'In Transit' } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
supply_chain_logs > |
```

## DELETE

```
db.shipment_logs.deleteOne({ order_id: 4 })
```

```
}  
> db.shipment_logs.deleteOne({ order_id: 4 })  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
supply_chain_logs > |
```

## 5. Create indexes for efficient querying

### Index for Efficient Querying

```
db.shipment_logs.createIndex({ order_id: 1 })  
db.shipment_logs.getIndexes()
```

```
> db.shipment_logs.createIndex({ order_id: 1 })  
< order_id_1  
> db.shipment_logs.getIndexes()  
< [  
  { v: 2, key: { _id: 1 }, name: '_id_' },  
  { v: 2, key: { order_id: 1 }, name: 'order_id_1' }  
]  
supply_chain_logs>
```

## Week 2 – Data Collection & Preprocessing in Python

### Tools: Python (Pandas, NumPy)

1. Use requests to fetch data from a sample API or local JSON/CSV

#### Sample CSV File: orders.csv

```
order_id,supplier_id,delivery_date,stock_quantity  
1,101,2026-05-01,50  
2,102,2026-05-05,20  
3,103,2026-05-10,15  
4,104,2026-05-07,30  
5,105,2026-05-03,10
```

2. Clean data using pandas (drop nulls, format dates, etc.)
3. Perform basic calculations using numpy (e.g., delays, stock levels)
4. Display the cleaned and processed data

## Python Code

```
import pandas as pd
import numpy as np
```

```
# Load CSV data
df = pd.read_csv('orders.csv')
print('Original Data:')
print(df)
```

```
... Original Data:
   order_id  supplier_id delivery_date  stock_quantity
0         1          101   2026-05-01             50
1         2          102   2026-05-05             20
2         3          103   2026-05-10             15
3         4          104   2026-05-07             30
4         5          105   2026-05-03             10
```

```
# Convert delivery_date to datetime
df['delivery_date'] = pd.to_datetime(df['delivery_date'])
```

```
# Remove null values
df = df.dropna()
```

```
# Calculate delay days
df['delay_days'] = (pd.Timestamp.today() -
df['delivery_date']).dt.days
```

```
# Create delayed flag using numpy
df['is_delayed'] = np.where(df['delay_days'] > 0, 1, 0)
```

```
print('\nProcessed Data:')
print(df)
```

```
... Processed Data:
   order_id  supplier_id delivery_date  stock_quantity  delay_days  is_delayed
0         1          101   2026-05-01             50           13           1
1         2          102   2026-05-05             20           9           1
2         3          103   2026-05-10             15           4           1
3         4          104   2026-05-07             30           7           1
4         5          105   2026-05-03             10          11           1
```

## Week 3 – Intro to PySpark: Processing Big Data

### Tools: PySpark

1. Load order data from CSV using PySpark

#### Sample CSV File: orders.csv

```
order_id,supplier_id,delivery_date,is_delayed
1,101,2026-05-01,1
2,102,2026-05-05,1
3,101,2026-05-10,0
4,103,2026-05-07,1
5,102,2026-05-03,1
6,101,2026-05-08,0
7,103,2026-05-09,1
```

2. Filter delayed shipments
3. Group by supplier and count delayed orders
4. Save processed data to CSV or Parquet

### PySpark Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

# Create Spark session
spark = SparkSession.builder \
    .appName('SupplyChainProcessing') \
    .getOrCreate()

# Load CSV data
df = spark.read.csv('orders.csv', header=True, inferSchema=True)
print('Original Data:')
df.show()
```

Original Data:

| order_id | supplier_id | delivery_date | is_delayed |
|----------|-------------|---------------|------------|
| 1        | 101         | 2026-05-01    | 1          |
| 2        | 102         | 2026-05-05    | 1          |
| 3        | 101         | 2026-05-10    | 0          |
| 4        | 103         | 2026-05-07    | 1          |
| 5        | 102         | 2026-05-03    | 1          |
| 6        | 101         | 2026-05-08    | 0          |
| 7        | 103         | 2026-05-09    | 1          |

```
# Filter delayed shipments
delayed_df = df.filter(col('is_delayed') == 1)
print('Delayed Shipments:')
delayed_df.show()
```

```
... Delayed Shipments:
+-----+-----+-----+-----+
|order_id|supplier_id|delivery_date|is_delayed|
+-----+-----+-----+-----+
|      1|      101|  2026-05-01|         1|
|      2|      102|  2026-05-05|         1|
|      4|      103|  2026-05-07|         1|
|      5|      102|  2026-05-03|         1|
|      7|      103|  2026-05-09|         1|
+-----+-----+-----+-----+
```

```
# Group by supplier and count delayed orders
grouped_df = delayed_df.groupBy('supplier_id') \
    .agg(count('order_id').alias('delayed_order_count'))
print('Grouped Results:')
grouped_df.show()
```

```
... Grouped Results:
+-----+-----+
|supplier_id|delayed_order_count|
+-----+-----+
|      101|                1|
|      103|                2|
|      102|                2|
+-----+-----+
```

```
# Save output to CSV
grouped_df.write.mode('overwrite') \
    .csv('output/grouped_results', header=True)
```

```
# Save output to Parquet (added)
grouped_df.write.mode('overwrite') \
    .parquet('output/grouped_results_parquet')
```

```
print('Data saved to CSV and Parquet successfully.')
# Stop Spark session
spark.stop()
```



The image shows two side-by-side screenshots from a data science environment. The left screenshot shows a file explorer with a folder named 'output' containing two files: 'grouped\_results' and 'grouped\_results\_parquet'. The right screenshot shows a data table view for a file named 'part-00000-a57d927e-a12d-4f9f-9d29-'. The table has two columns: 'supplier\_id' and 'delayed\_order\_count'. It displays 3 entries: supplier\_id 101 with count 1, supplier\_id 103 with count 2, and supplier\_id 102 with count 2. The table is titled '1 to 3 of 3 entries' and has a 'Filter' button. Below the table, it says 'Show 10 per page'.

| supplier_id | delayed_order_count |
|-------------|---------------------|
| 101         | 1                   |
| 103         | 2                   |
| 102         | 2                   |

## Week 4 – Simple ETL in Azure Databricks

### Tools: Azure Databricks

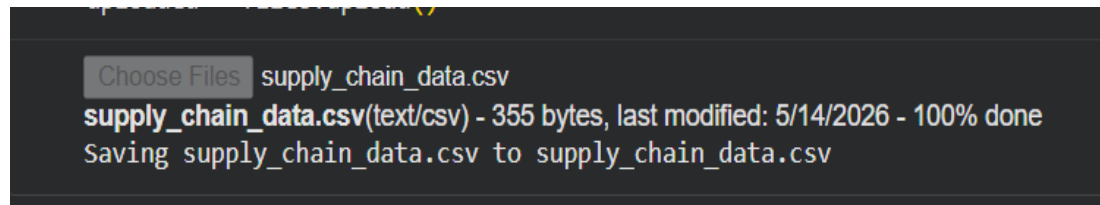
#### Notebook Name: week4\_etl\_notebook

##### 1. Upload CSV data into Databricks

```
from pyspark.sql import SparkSession
```

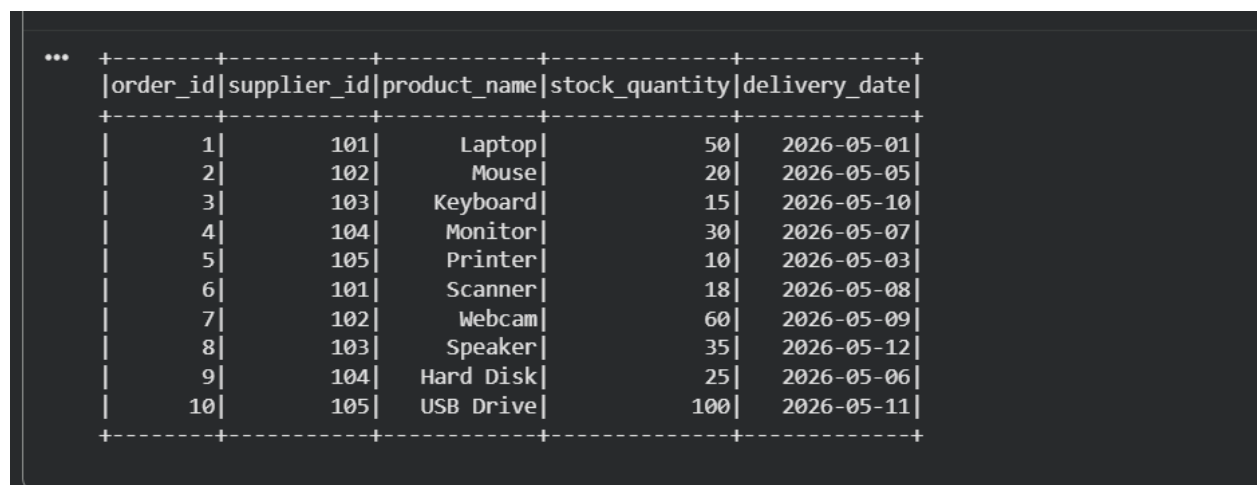
```
spark =  
SparkSession.builder.appName("SupplyChainMonitoring").getOrCreate()
```

```
from google.colab import files  
uploaded = files.upload()
```



```
supply_df = spark.read.csv(  
    "supply_chain_data.csv",  
    header=True,  
    inferSchema=True  
)
```

```
supply_df.show()
```



| order_id | supplier_id | product_name | stock_quantity | delivery_date |
|----------|-------------|--------------|----------------|---------------|
| 1        | 101         | Laptop       | 50             | 2026-05-01    |
| 2        | 102         | Mouse        | 20             | 2026-05-05    |
| 3        | 103         | Keyboard     | 15             | 2026-05-10    |
| 4        | 104         | Monitor      | 30             | 2026-05-07    |
| 5        | 105         | Printer      | 10             | 2026-05-03    |
| 6        | 101         | Scanner      | 18             | 2026-05-08    |
| 7        | 102         | Webcam       | 60             | 2026-05-09    |
| 8        | 103         | Speaker      | 35             | 2026-05-12    |
| 9        | 104         | Hard Disk    | 25             | 2026-05-06    |
| 10       | 105         | USB Drive    | 100            | 2026-05-11    |

## 2. Run a notebook to clean and filter the data

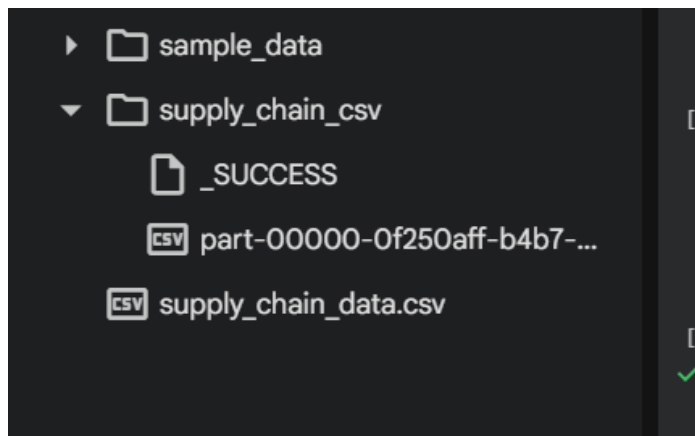
```
from pyspark.sql.functions import current_date, datediff, when

# Convert delivery_date column to date type
supply_df = supply_df.withColumn(
    "delivery_date",
    supply_df["delivery_date"].cast("date")
)
# Calculate delay days
supply_df = supply_df.withColumn(
    "delay_days",
    datediff(current_date(), supply_df["delivery_date"])
)
# Add delayed status column
supply_df = supply_df.withColumn(
    "shipment_status",
    when(supply_df.delay_days > 0, "Delayed")
    .otherwise("On Time")
)
# Filter delayed shipments
delayed_df = supply_df.filter(
    supply_df.shipment_status == "Delayed"
)
delayed_df.show()
```

```
... +-----+-----+-----+-----+-----+-----+-----+
|order_id|supplier_id|product_name|stock_quantity|delivery_date|delay_days|shipment_status|
+-----+-----+-----+-----+-----+-----+-----+
|1|101|Laptop|50|2026-05-01|14|Delayed|
|2|102|Mouse|20|2026-05-05|10|Delayed|
|3|103|Keyboard|15|2026-05-10|5|Delayed|
|4|104|Monitor|30|2026-05-07|8|Delayed|
|5|105|Printer|10|2026-05-03|12|Delayed|
|6|101|Scanner|18|2026-05-08|7|Delayed|
|7|102|Webcam|60|2026-05-09|6|Delayed|
|8|103|Speaker|35|2026-05-12|3|Delayed|
|9|104|Hard Disk|25|2026-05-06|9|Delayed|
|10|105|USB Drive|100|2026-05-11|4|Delayed|
+-----+-----+-----+-----+-----+-----+-----+
```

3. Save cleaned output as Delta or CSV

```
delayed_df.write.mode("overwrite").csv(
    "supply_chain_csv",
    header=True
)
saved_df = spark.read.csv(
    "supply_chain_csv",
    header=True,
    inferSchema=True
)
saved_df.show()
```



| part-00000-0f250aff-b4b7-40a2-b377-c8f2213144f8-c000.csv |             |              |                |               |            |                 |
|----------------------------------------------------------|-------------|--------------|----------------|---------------|------------|-----------------|
| 1 to 10 of 10 entries                                    |             |              |                |               |            |                 |
| order_id                                                 | supplier_id | product_name | stock_quantity | delivery_date | delay_days | shipment_status |
| 1                                                        | 101         | Laptop       | 50             | 2026-05-01    | 14         | Delayed         |
| 2                                                        | 102         | Mouse        | 20             | 2026-05-05    | 10         | Delayed         |
| 3                                                        | 103         | Keyboard     | 15             | 2026-05-10    | 5          | Delayed         |
| 4                                                        | 104         | Monitor      | 30             | 2026-05-07    | 8          | Delayed         |
| 5                                                        | 105         | Printer      | 10             | 2026-05-03    | 12         | Delayed         |
| 6                                                        | 101         | Scanner      | 18             | 2026-05-08    | 7          | Delayed         |
| 7                                                        | 102         | Webcam       | 60             | 2026-05-09    | 6          | Delayed         |
| 8                                                        | 103         | Speaker      | 35             | 2026-05-12    | 3          | Delayed         |
| 9                                                        | 104         | Hard Disk    | 25             | 2026-05-06    | 9          | Delayed         |
| 10                                                       | 105         | USB Drive    | 100            | 2026-05-11    | 4          | Delayed         |

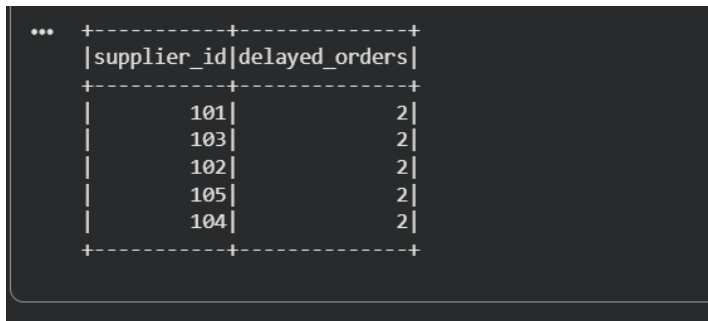


#### 4. Run basic analysis queries using SQL or PySpark

```
delayed_df.createOrReplaceTempView("supply_chain_orders")

result = spark.sql("""
SELECT supplier_id,
COUNT(*) AS delayed_orders
FROM supply_chain_orders
WHERE shipment_status = 'Delayed'
GROUP BY supplier_id
ORDER BY delayed_orders DESC
""")

result.show()
```



| supplier_id | delayed_orders |
|-------------|----------------|
| 101         | 2              |
| 103         | 2              |
| 102         | 2              |
| 105         | 2              |
| 104         | 2              |

### Week 5 – Deploy & Automate with Azure DevOps

#### Tools: Azure DevOps

Tasks:

##### 1. Create a simple Azure DevOps pipeline to run Python scripts

trigger:

- main

pool:

vmImage: 'ubuntu-latest'

steps:

- script: |  
    pip install -r requirements.txt  
    python run\_pipeline.py  
    displayName: 'Run Supply Chain Script'

## 2. Install dependencies and execute the project

**requirements.txt**

pandas

numpy

**run\_pipeline.py**

```
print("Starting Supply Chain Pipeline...")
```

```
print("Loading dataset...")
```

```
print("Processing inventory records...")
```

```
print("Generating analysis report...")
```

```
print("Pipeline Execution Completed Successfully")
```

## 3. Log the results and mark completion

### **Sample Console Output**

Collecting pandas

Collecting numpy

Successfully installed required packages

Starting Supply Chain Pipeline...

Loading dataset...

Processing inventory records...

Generating analysis report...

Pipeline Execution Completed Successfully